University of Wolverhampton
School of Mathematics and Computer Science

6CS005 High Performance Computing Week 6 Workshop

Tasks - OpenMP Multithreading

You may need to refer to the Week 6 lecture slides in order to complete these tasks.

1. The following program prints out "Hello World!" with the default number of threads which is usually the number of CPU processor cores that are on your computer system:

```
#include <stdio.h>
void main()
{
  #pragma omp parallel
  printf("Hello world from OpenMP!\n");
}
```

   a. Enter and run the program to see how many processor cores are on the system you are using.
   b. Modify the program so that it run with exactly 10 threads.

2. The following program prints out all the prime numbers from 1 to 1000:

```
#include <stdio.h>
void main()
{
  int i, c;
  printf("Prime numbers between 1 and 1000 are :\n");
  for(i = 1; i <= 1000; i++){
    for(c = 2; c <= i - 1; c++){
      if ( i % c == 0 )
        break;
    }
    if ( c == i )
      printf("%d\n", i);
  }
}
```

   Convert this to a multithread OpenMP program using the default number of threads to calculate the prime numbers.

3. Convert the above program so that it uses exactly 5 threads and each thread prints out their thread ID when they print out their prime number.

4. Modify the program in (3) so that the program prints out the total number of prime numbers found. Check this whether this is correct by running it with exactly 1 thread.

5. The following PThreads program demonstrates 3 thread sending string messages to each other, using a global array. The messages are meant to be sent in the following order:

   a. Thread 0 sends Thread 1 a message
   b. Thread 1 receives the message

c. Thread 1 sends Thread 2 a message
d. Thread 2 receives the message
e. Thread 2 sends Thread 0 a message
f. Thread 0 receives the message
g. This then repeats from (a) 10 times

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <unistd.h>

char *messages[3] = {NULL, NULL, NULL};

void *messenger(void *p)
{
  long tid = (long)p;
  char tmpbuf[100];

  for(int i=0; i<10; i++)
  {
    /* Sending a message */
    long int dest = (tid + 1) % 3;
    sprintf(tmpbuf,"Hello from Thread %ld!", tid);
    char *msg = strdup(tmpbuf);
    messages[dest] = msg;
    printf("Thread %ld sent the message to Thread %ld\n",tid, dest);

    /* Receiving a message */
    printf("Thread %ld received the message '%s'\n",tid, messages[tid]);
    free(messages[tid]);
    messages[tid] = NULL;
  }
  return NULL;
}

void main()
{
  pthread_t thrID1, thrID2, thrID3;

  pthread_create(&thrID1, NULL, messenger, (void *)0);
  pthread_create(&thrID2, NULL, messenger, (void *)1);
  pthread_create(&thrID3, NULL, messenger, (void *)2);
  pthread_join(thrID1, NULL);
  pthread_join(thrID2, NULL);
  pthread_join(thrID3, NULL);
}
```

Convert the program to using OpenMP instead of Pthreads.